

Aktualizacja i Modernizacja UX/UI TipJar+ (2025)

TipJar+ to nowoczesna platforma mikropłatności dla twórców treści, łącząca świat Web3 z przyjaznym interfejsem znanym z tradycyjnych aplikacji. Celem poniższego poradnika jest przedstawienie zaktualizowanego projektu UX/UI TipJar+ zgodnego z najnowszymi trendami (m.in. Web3, stablecoiny, DAO/SocialFi), zachowującego funkcjonalność, elegancję i prostotę – bez zbędnych fajerwerków. Wszystkie komponenty zostały zaprojektowane z myślą o ciemnym motywie (dark mode) z przewagą turkusu i złota, oraz delikatnymi akcentami purpury, a także spełniają wymogi dostępności WCAG (kontrast kolorów, czytelna typografia). Efektem końcowym jest kompletny front-end gotowy do wdrożenia: zestaw plików kodu React (Next.js) z użyciem TypeScript i Tailwind CSS, zorganizowany w czytelnej strukturze projektowej. Poniżej znajduje się opis architektury frontendu, kluczowych decyzji projektowych oraz fragmenty kodu dla głównych ekranów i komponentów aplikacji, uzupełniony o krótkie instrukcje integracyjne (README) dla deweloperów.

Stos technologiczny i biblioteki

Projekt wykorzystuje nowoczesny stos technologiczny, zapewniający wydajność, bezpieczeństwo i łatwą integrację z elementami Web3:

Next.js 13+ (React) – Framework frontendu (TypeScript) zapewniający hybrydowy model renderowania (SSR/SSG) oraz doskonały system routingu. W projekcie wykorzystano App Router Next.js z layoutami dla spójnego UI. Routing uwzględnia dynamiczne ścieżki (np. profile twórców jako `/[username]`) zgodnie z wymaganiami.

Tailwind CSS – Utility-first CSS framework do stylowania. Projekt zawiera rozszerzoną konfigurację Tailwind (w pliku `tailwind.config.js`) z zdefiniowaną paletą kolorów marki TipJar+ oraz wsparciem dla trybu ciemnego/jasnego (`darkMode: 'class'`) i pluginami potrzebnymi do integracji z bibliotekami UI.

@headlessui/react – Biblioteka bezstylowych, dostępnych komponentów UI (np. modale, menu), używana np. do tworzenia responsywnych dialogów i rozwijanych menu zgodnych z WCAG.

shadcn/ui – Zestaw współpracujących z Tailwind komponentów UI (opartych o Radix UI) zapewniający spójny wygląd i zachowanie elementów takich jak przyciski, pola formularzy, okna dialogowe itp. Użycie shadcn/ui przyspiesza tworzenie interfejsu zachowując najlepsze praktyki design systemu.

lucide-react – Ikony wektorowe w React (następca Feather Icons). W projekcie używane do ikon akcji (np. ikona menu, ikony sieci społecznościowych, portfela itp.), zapewniając nowoczesny, liniowy styl pasujący do minimalistycznego designu.

framer-motion – Biblioteka do animacji w React. Służy do implementacji subtelnych animacji przejść między widokami, pojawiania się okien modalnych czy hover efektów na przyciskach, zgodnie z założeniem dynamicznego, ale nienachalnego UI.

zustand – Lekka biblioteka do zarządzania globalnym stanem aplikacji. W TipJar+ Zustand przechowuje stan użytkownika (informacje o zalogowaniu, dane profilu) oraz ustawienia globalne (np. tryb jasny/ciemny), zapewniając wydajność i prostotę API.

wagmi + viem (Ethereum) – Biblioteki do integracji z portfelami kryptowalut (Ethereum). Wagmi umożliwia łatwe podłączenie portfela (np. MetaMask) i obsługę podpisywania wiadomości (np. SIWE – Sign-In with Ethereum), natomiast viem służy do komunikacji z blockchainem (wykorzystywany np. przy wywołaniach on-chain, choć większość transakcji w TipJar odbywa się off-chain poprzez API Circle).

(Opcjonalnie) NextAuth.js – Rozważane do obsługi OAuth (Google, Twitch), choć w obecnej implementacji integracja z OAuth i SIWE jest obsługiwana przez własne endpointy backendu (NestJS + Passport JWT). Frontend zapewnia UI dla tych metod logowania, niezależnie od konkretnej implementacji zaplecza.

Design system: paleta kolorów, typografia i styl

Kolorystyka: Zgodnie z przewodnikiem marki TipJar+, dominuje ciemny turkus (#003737) jako kolor tła aplikacji i głównych segmentów UI. Elementy interaktywne (przyciski, ikony akcji) wyróżniono kolorem złotym (#FFD700) – to barwa akcentowa przyciągająca uwagę na ciemnym tle. Dla zapewnienia odpowiedniego kontrastu tekst jest biały (#FFFFFF) lub w odcieniach bardzo jasnoszarych. Dodatkowo w wybranych detalach pojawia się purpura (#7E3FF2) – subtelnie podkreślając niektóre elementy (np. aktywne stany przycisków, linki lub tło w sekcjach specjalnych). Ważne jest zachowanie zgodności z WCAG AA – kontrast między złotym tekstem/ikoną a turkusowym tłem musi być wystarczający, dlatego np. złote przyciski mają odpowiednią ciemniejszą obwódkę lub cień, by poprawić czytelność na tle.

Typografia: W całej aplikacji wykorzystano font Montserrat (bezseryfowy, nowoczesny) – nagłówki w wariantcie Bold dla mocnego akcentu, a teksty paragrafów i interfejsu w wariantcie Regular. Font Montserrat zapewnia profesjonalny wygląd i dobrą czytelność na ekranie. Jako fallback zdefiniowano ogólną rodzinę sans-serif (systemową) na wypadek problemów z wczytaniem Montserrata. Wielkości fontów i wysokości linii zostały dobrane pod kątem czytelności na różnych rozdzielczościach, a cała typografia została przetestowana pod kątem dostępności (odpowiedni kontrast i skalowalność z zoomem przeglądarki).

Styl graficzny: Interfejs jest nowoczesny i minimalistyczny – dominuje płaskie projektowanie (flat design) z ograniczeniem zbędnych dekoracji. Ciemnoturkusowe tła i złote akcenty tworzą elegancki kontrast. Unikamy efektów takich jak ciężkie cienie, zbędne gradienty czy nadmiar animacji – prostota zwiększa czytelność i profesjonalizm platformy. Wszelkie efekty graficzne (np. niewielkie cienie, delikatne obramowania przy kartach) są użyte oszczędnie i służą jedynie wyróżnieniu ważnych elementów, nie odwracając uwagi od głównej treści. Ikony (z pakietu lucide-react) są proste, liniowe, dopasowane stylowo do fontu Montserrat.

Dostępność: Projekt został zrealizowany zgodnie ze standardem WCAG 2.1 AA – zadbane o odpowiedni kontrast kolorów (jasny tekst na ciemnym tle, czytelne złote elementy na tle turkusowego). Wszystkie interaktywne komponenty (linki, przyciski, pola formularzy) posiadają

stany focus/hover dostosowane dla użytkowników korzystających z klawiatury lub technologii asystujących. Użycie biblioteki Headless UI pomaga zapewnić właściwe zachowanie komponentów dialogowych i nawigacyjnych pod kątem dostępności (np. focus trap w modalu, oznaczenia ARIA). Całość interfejsu jest również responsywna – układy zostały zaprojektowane w podejściu mobile-first i testowane na różnych rozdzielczościach (od małych ekranów telefonów po duże ekrany desktopowe). Layout dynamicznie dostosowuje się, np. menu boczne dashboardu staje się chowanym off-canvas na urządzeniach mobilnych, a widoki kart zmieniają się w listy na wąskich ekranach.

Branding: Logo TipJar+ to stylizowany zarys słoika (symbol napiwków) z dodanym znakiem “+”, w kolorze złotym na tle ciemnoturkusowym. Logo występuje w aplikacji w postaci ikony (np. na pasku nawigacji jako znak rozpoznawczy) oraz pełnej nazwy obok (np. na stronie głównej). Poniżej przedstawiono przykładową ikonę logo w wersji ciemnego motywu:

Logo TipJar+ – złoty symbol słoika z “+” na ciemnoturkusowym tle (wersja dark mode).

Struktura projektu i organizacja kodu

Projekt został zorganizowany w czytelny sposób, ułatwiający dalszy rozwój i utrzymanie kodu. Poniżej przedstawiono uproszczoną strukturę plików frontendu (Next.js + TypeScript):

```
tipjar-plus-frontend/
├── app/ (Next.js App Router - jeśli używamy App Directory)
│   ├── layout.tsx    # globalny layout aplikacji (np. <html> z motywem)
│   ├── page.tsx      # strona główna (Landing Page)
│   ├── dashboard/    # zagnieżdżone podstrony panelu twórcy
│   │   ├── layout.tsx # layout panelu twórcy (sidebar, header)
│   │   ├── page.tsx   # domyślna strona dashboardu (np. przekierowanie lub przegląd)
│   │   ├── stats/page.tsx # statystyki napiwków (4.3.3.1)
│   │   ├── profile/page.tsx # ustawienia profilu twórcy (4.3.3.2)
│   │   ├── withdrawals/page.tsx # opcje wypłat (4.3.3.3)
│   │   └── ... (inne sekcje dashboardu)
│   ├── profile/[username]/page.tsx # publiczny profil twórcy (dynamiczny routing)
│   ├── auth/          # strony rejestracji/logowania
│   │   ├── register/page.tsx # rejestracja
│   │   └── login/page.tsx   # logowanie (jeśli rozdzielone)
│   ├── _document.tsx   # ustawienia dokumentu (np. wczytanie fontów Montserrat)
│   └── globals.css     # import Tailwind CSS i globalne style
├── components/         # współdzielone komponenty UI
│   ├── Navbar.tsx     # nawigacja górna (logo, przyciski, przełącznik trybu)
│   ├── Footer.tsx     # stopka strony publicznej
│   ├── CreatorCard.tsx # komponent karty twórcy (używany na stronie odkrywania itp.)
│   ├── TipButton.tsx  # przycisk "Wesprzyj" z logiką otwarcia modala
│   ├── TipModal.tsx   # modal przekazywania napiwku (formularz płatności)
│   ├── TipHistory.tsx # lista ostatnich napiwków (do profilu twórcy, statystyk)
│   ├── DashboardSidebar.tsx # boczna nawigacja w panelu twórcy
│   ├── DashboardHeader.tsx # nagłówek w panelu twórcy (np. avatar twórcy, menu)
│   └── StatsChart.tsx  # wykres do statystyk (np. biblioteka Chart.js lub inna)
```

- | — ... (inne komponenty jak FormularzWypłaty, Listafiltrów itp.)
- | — hooks/ # niestandardowe hooki
 - | — useThemeToggle.ts # logika przełączania dark/light mode (np. z localStorage)
 - | — useAuth.ts # zarządzanie stanem autentykacji (np. sprawdzanie JWT)
 - | — useUserProfile.ts # pobieranie i cachowanie danych profilu użytkownika
- | — store/ # stan globalny (zustand)
 - | — useStore.ts # definicja store (np. { user, setUser, theme, setTheme, ... })
- | — lib/ # moduły pomocnicze, integracje
 - | — api.ts # funkcje do komunikacji z backend API (fetch/axios)
 - | — wagmi.ts # konfiguracja wagmi (connectors, provider, chains)
 - | — ... (np. utils formatowania dat/kwot)
- | — public/ # zasoby statyczne (obrazki, ikony, czcionki)
 - | — logo.svg # plik wektorowy logo
- | — tailwind.config.js # konfiguracja Tailwind (kolory marki, breakpoints)
- | — tsconfig.json # konfiguracja TypeScript
- | — README.md # dokumentacja uruchomienia i integracji

W powyższej strukturze zastosowano podejście modułowe: strony (app/*) odpowiadają głównym widokom aplikacji, a wewnątrz komponentów i hooków znajduje się logika, którą można reużyć na wielu stronach. Dzięki Next.js (App Router) możliwe jest definiowanie layoutów – np. globalny layout aplikacji (dla wszystkich stron, zawierający <head> i import fontów) oraz osobny layout dla sekcji /dashboard (zawierający np. wspólny sidebar i header dla panelu twórcy). To ułatwia utrzymanie spójności UI i pozwala na implementację tzw. nested routing, czyli zagnieżdżonych tras w panelu twórcy bez przeładowywania całej strony.

Konfiguracja Tailwind: W pliku tailwind.config.js dodano niestandardowe kolory dla marki TipJar+, aby móc łatwo stosować je w klasach Tailwind. Przykład fragmentu konfiguracji:

```
// tailwind.config.js (fragment)
module.exports = {
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        // Kolory brandowe TipJar+
        brand: {
          dark: '#003737', // główny ciemny turkus
          gold: '#FFD700', // akcent złoty
          purple: '#7E3FF2', // akcent purpura
          // ewentualnie dodatkowe odcienie:
          light: '#00FFEF', // jasny turkus (np. hover)
        }
      },
    },
    fontFamily: {
      sans: ['Montserrat', 'sans-serif']
    }
  }
}
```

```

},
plugins: [
  require('@headlessui/tailwindcss'), // opcjonalnie plugin headlessui jeśli wymagany
  require('@tailwindcss/forms')      // stylowanie elementów formularzy
  // ...inne pluginy
]
}

```

Dzięki takiej konfiguracji w kodzie możemy używać klas Tailwind odwołujących się do kolorów, np. `bg-brand-dark`, `text-brand-gold`, co zapewnia spójność kolorystyczną z przewodnikiem. Tryb dark mode jest obsługiwany poprzez dodanie/odjęcie klasy `dark` na elemencie `<html>` – korzystamy tu z mechanizmu preferencji użytkownika lub przełącznika w interfejsie (opisany dalej). Tailwind automatycznie generuje warianty dark: dla klas, więc np. `dark:bg-white` zmieni kolor tła po przełączeniu motywu.

Implementacja UI – główne ekrany i komponenty

Poniżej omówiono kluczowe ekrany aplikacji TipJar+ oraz przedstawiono przykładowe implementacje w kodzie (React + Next.js + Tailwind). Każdy ekran został zaadaptowany zgodnie z wymaganiami funkcjonalnymi z oryginalnego poradnika, ale uwspółcześniony pod kątem designu i technologii (np. dodano obsługę najnowszych bibliotek, zapewniono pełną responsywność oraz integrację z Web3). Kody zawierają placeholdery (np. przykładowe dane, uproszczoną logikę), aby były od razu uruchamialne – zakłada się, że integracja z backendem (NestJS) nastąpi poprzez wywołania API w zaznaczonych miejscach.

Strona Główna (Landing Page)

Strona główna jest wizytówką platformy – ma za zadanie przedstawić ideę TipJar+ i zachęcić do rejestracji. Projekt przewiduje duży, atrakcyjny nagłówek (hero section) z hasłem i przyciskami CTA, a poniżej sekcje informacyjne dla twórców i fanów. Wszystko utrzymane w ciemnej tonacji z kontrastującymi złotymi akcentami.

Kluczowe elementy strony głównej:

Nagłówek (Hero): Krótkie hasło wyjaśniające funkcję platformy (np. „Wspieraj ulubionych twórców bez granic za pomocą kryptowaluty USDC”) oraz przyciski CTA: dla twórcy „Załącz profil” (link do rejestracji) i dla fana „Znajdź twórcę” (scroll do sekcji listy twórców lub przekierowanie do strony Odkrywaj). W tle można dodać ilustrację (np. globalnej sieci łączącej twórców i fanów) podkreślającą przekaz.

Sekcja „Jak to działa / Korzyści”: Ikony + krótkie opisy głównych zalet TipJar+ dla użytkownika. Przykładowo: Niskie prowizje (TipJar+ pobiera ~7% vs 20-30% u konkurencji), Globalny zasięg (wpłaty w USDC z dowolnego miejsca na świecie), Błyskawiczne wypłaty (dzięki integracji z Circle – natychmiastowe transfery wewnętrzne), Prostota integracji (widget lub link dla twórcy do udostępnienia).

Sekcja dla Twórców: Krótki opis jak twórcy mogą skorzystać (np. „Założ konto, udostępni link fanom, odbieraj napiwki w kryptowalucie bez prowizji banków”) plus przycisk “Zarejestruj się” prowadzący do rejestracji twórcy.

Sekcja dla Fanów: Opis dla fanów (np. „Wspieraj ulubionych twórców już od \$1, bez potrzeby zakładania konta”). Można tu podkreślić, że fan nie musi zakładać konta by wysłać napiwek (możliwość działania jako gość), oraz dać link “Odkryj twórców” do listy profili.

Stopka: Na dole strony – zawiera linki informacyjne (O nas, Regulamin, Kontakt) oraz ikonki social mediów TipJar+.

Poniżej znajduje się uproszczona implementacja strony głównej w Next.js (TypeScript). Wykorzystano tu komponenty: Navbar, Footer (współdzielone), a sekcje zrealizowano przy pomocy prostych komponentów i stylów Tailwind. Przyciski CTA korzystają z klas shadcn/ui dla spójnego stylu (np. btn z odpowiednimi stylami) oraz ikon z lucide-react jeśli potrzebne. Fragmenty tekstów wzięte zostały z oryginalnego poradnika jako placeholderzy treści.

```
// app/page.tsx - Landing Page (Strona Główna)
import { Button } from "@components/ui/button" // przykładowy import z shadcn/ui
import { ArrowRight } from "lucide-react"      // ikona strzałki z lucide-react

export default function LandingPage() {
  return (
    <main className="min-h-screen bg-brand-dark text-white flex flex-col">
      {/* Navbar */}
      <Navbar />

      {/* Hero Section */}
      <section className="flex flex-col items-center justify-center flex-1 px-4 py-12 text-center">
        <h1 className="text-4xl md:text-5xl font-bold mb-6">
          Wspieraj ulubionych twórców bez granic, za pomocą kryptowaluty USDC
        </h1>
        <p className="text-lg md:text-2xl text-gray-100 mb-8">
          TipJar+ to platforma napiwków dla twórców treści – nowoczesna (Web3), a jednocześnie prosta jak tradycyjne aplikacje.
        </p>
        <div className="space-x-4">
          <Button asChild className="bg-brand-gold text-black font-semibold px-6 py-3">
            <a href="/auth/register">🎨 Załóż profil twórcy</a>
          </Button>
          <Button variant="outline" asChild className="text-brand-gold border-brand-gold px-6 py-3">
            <a href="#discover">🔍 Znajdź twórcę</a>
          </Button>
        </div>
      </section>
    </main>
  )
}
```

```

    { /* How it works / Benefits Section */ }
    <section id="benefits" className="py-16 bg-brand-dark/80">
      <h2 className="text-3xl font-bold text-center mb-12 text-brand-gold">Jak to
działa?</h2>
      <div className="max-w-5xl mx-auto grid grid-cols-1 md:grid-cols-3 gap-8 px-6
text-center">
        <div className="flex flex-col items-center">
          <ArrowRight className="w-12 h-12 text-brand-gold mb-4" />
          <h3 className="text-xl font-semibold mb-2">Niskie prowizje</h3>
          <p className="text-gray-200">Tylko ~7% opłat (razem) vs 20-30% na innych
platformach. Więcej wsparcia trafia do twórcy!33</p>
        </div>
        <div className="flex flex-col items-center">
          <ArrowRight className="w-12 h-12 text-brand-gold mb-4" />
          <h3 className="text-xl font-semibold mb-2">Globalny zasięg</h3>
          <p className="text-gray-200">Wspieraj i zarabiaj z każdego miejsca na świecie
dzięki USDC – szybkie, bez granic34.</p>
        </div>
        <div className="flex flex-col items-center">
          <ArrowRight className="w-12 h-12 text-brand-gold mb-4" />
          <h3 className="text-xl font-semibold mb-2">Błyskawiczne wypłaty</h3>
          <p className="text-gray-200">Środki trafiają natychmiast na portfel twórcy. Wypłaty
on-chain realizowane w ciągu minut (Circle API).</p>
        </div>
      </div>
    </section>

```

```

    { /* Section for Creators */ }
    <section className="py-16 bg-brand-dark">
      <div className="max-w-4xl mx-auto px-6 text-center md:text-left">
        <h2 className="text-2xl font-bold text-brand-gold mb-6">Dla Twórców</h2>
        <p className="text-gray-100 mb-4">
          Załóż konto, udostępnij link fanom, odbieraj napiwki w kryptowalucie bez prowizji
banków35. TipJar+ automatycznie utworzy dla Ciebie portfel w USDC i zajmie się resztą!
        </p>
        <Button asChild className="bg-brand-gold text-black font-medium px-5 py-3">
          <a href="/auth/register">Zarejestruj się jako Twórca</a>
        </Button>
      </div>
    </section>

```

```

    { /* Section for Fans */ }
    <section className="py-16 bg-brand-dark/90">
      <div className="max-w-4xl mx-auto px-6 text-center md:text-left">
        <h2 className="text-2xl font-bold text-brand-gold mb-6">Dla Fanów</h2>
        <p className="text-gray-100 mb-4">

```

Wspieraj swoich ulubionych twórców już od \$1 – nawet ****bez zakładania konta****. Po prostu wybierz twórcę i wyślij napiwek, to proste!

```
</p>
<Button asChild variant="outline" className="text-brand-gold border-brand-gold
font-medium px-5 py-3">
  <a href="/profile/demo#tip">Odkryj Twórców</a>
</Button>
</div>
</section>

{ /* Footer */ }
<Footer />
</main>
);
}
```

Wyjaśnienia do powyższego kodu: Zastosowano semantyczne sekcje HTML (<section>) dla logicznego podziału strony. Każda sekcja ma własne tło (niektóre wykorzystują przezroczystość koloru, np. bg-brand-dark/80, by delikatnie odróżnić się warstwowo). Tekst nagłówkowy i przyciski używają koloru złotego dla podkreślenia – np. klasa text-brand-gold dla nagłówka sekcji lub złoty przycisk CTA dla twórców. Użyto gotowego komponentu <Button> z shadcn/ui, który domyślnie posiada styl przycisku; nadpisano go częściowo własnymi klasami Tailwind (className) by zastosować kolory brandowe. Dodatkowo, pokazano użycie ikon lucide-react (tutaj symbolicznie wykorzystano tę samą ikonę ArrowRight dla uproszczenia, w praktyce można użyć różnych ikon dla różnych korzyści). Teksty paragrafów zostały zaczerpnięte z dokumentacji (stąd np. cytaty w komentarzu z ID źródła). Cała sekcja hero jest wyśrodkowana i używa flex do centrowania treści, co zapewnia ładne rozmieszczenie na różnych rozmiarach ekranu. Dzięki klasom responsywnym (np. md:text-5xl) nagłówki zwiększa się na większych ekranach.

Nota UX: Strona główna komunikuje unikalną propozycję wartości TipJar+ – nowoczesność (Web3) połączoną z łatwością użycia tradycyjnej aplikacji. Elementy są rozmieszczone intuicyjnie: najpierw zainteresowanie hasłem, potem szybka informacja jak to działa i co zyskuje użytkownik. Design unika nadmiernej grafiki – ewentualna ilustracja w tle hero jest stonowana (np. ciemny, półprzezroczysty wzór), by nie rozpraszać od tekstu. Konsekwentnie stosowane są kolory brandowe. Wszystkie CTA są wyraźne i dostępne (duży kontrast, duże klikowalne przyciski również na mobile).

Publiczny Profil Twórcy (Widok dla Fana)

Strona profilu twórcy to centralne miejsce interakcji fana z twórcą. Musi prezentować najważniejsze informacje o twórcy i umożliwiać wysłanie napiwku przy minimalnej liczbie kroków. Profil jest publiczny (dostępny także dla niezalogowanych fanów), więc pełni rolę swoistej landing page dla twórcy – powinien budzić zaufanie i zachęcać do wsparcia.

Kluczowe elementy profilu twórcy:

Banner i Avatar: Górna część profilu zawiera baner (duża grafika w tle, personalizowana przez twórcę, np. zdjęcie tematyczne) oraz zdjęcie profilowe (avatar twórcy, np. okrągłe) wyświetlane na tle banera. Obok (lub poniżej na mobile) avatara pojawia się nazwa twórcy i jego krótki opis/bio.

Cel zbiórki (Goal): Jeśli twórca ustawił cel (np. „Zbieram na nowy mikrofon – 40% osiągnięte”), to pod bio wyświetla się pasek postępu celu.

Przycisk “Wesprzyj”: Najważniejszy element CTA – widoczny, duży złoty przycisk “Wesprzyj”. Kliknięcie go otwiera modal z interfejsem płatności (przepływ wysyłania napiwku, omówiony w następnym podrozdziale). Przyciski CTA są zaprojektowane tak, by zawsze były dostępne (również przy przewijaniu – np. można zastosować sticky bar z przyciskiem na dole ekranu mobilnego, aby fan nie musiał scrollować w poszukiwaniu sposobu wsparcia).

Adres do wpłaty (alternatywa): Obok przycisku “Wesprzyj” znajduje się ikona (np. portfela lub kodu QR) oznaczająca opcję uzyskania adresu portfela. Po kliknięciu otwiera się modal z publicznym adresem portfela twórcy (oraz kodem QR), co pozwala zaawansowanym użytkownikom dokonać wpłaty poza aplikacją bezpośrednio na adres (np. z własnego portfela). Ta opcja jest ukryta w ikonę, by nie rozpraszać mniej zaawansowanych fanów, ale dostępna dla chętnych.

Opis twórcy: Rozszerzony opis/bio twórcy, gdzie może on przedstawić siebie, wymienić osiągnięcia, dodać linki do swoich kanałów (YouTube, Twitch itd.). Ta sekcja daje kontekst fanowi, dlaczego warto go wesprzeć.

Historia wsparcia: Lista ostatnich napiwków otrzymanych przez twórcę – pokazuje np. pseudonim fana (lub “Anonimowy”), kwotę (np. \$5), datę/godzinę oraz ewentualną wiadomość od fana. Stanowi to społeczny dowód słuszności (social proof) – widząc, że inni fani wspierają twórcę, kolejni są bardziej skłonni to zrobić.

Komentarze / Tablica wsparcia: Ewentualnie sekcja krótkich publicznych podziękowań/wiadomości od fanów (zintegrowana z historią wpłat lub oddzielna) – planowana opcjonalnie jako rozwinięcie społecznościowego aspektu wsparcia.

Przyciski interakcji społecznościowej: W profilu można też uwzględnić przyciski typu “Obserwuj” twórcę (jeśli platforma wspiera obserwowanie) lub wyświetlać liczbę obserwujących/wspierających, ale w MVP skupiamy się na podstawowej funkcji wsparcia.

Poniżej przedstawiono uproszczoną implementację strony profilu twórcy (`app/profile/[username]/page.tsx`). Kod zakłada, że pobieramy dane twórcy (np. z API lub statycznie podczas build – tu dla uproszczenia wprost w komponencie). Wykorzystujemy komponenty: `TipButton` (który wewnątrz może otwierać `<TipModal>`), komponent do wyświetlania historii wsparcia (`TipHistory`) oraz ewentualnie komponent celu (progress bar). Dzięki dynamicznemu routingu Next.js, strona jest dostępna pod adresem np. `/profile/jan-kowalski` lub aliasie wybranym przez twórcę.

```

// app/profile/[username]/page.tsx - Public Creator Profile
import Image from 'next/image'
import { TipButton } from '@components/TipButton'
import { TipModal } from '@components/TipModal'
import { TipHistory } from '@components/TipHistory'
import { useState } from 'react'
import { QRCodeIcon } from 'lucide-react' // ikona dla adresu portfela (przykładowo)

interface Tip {
  id: number;
  from: string;
  amount: number;
  message?: string;
  date: string;
}

// Typ danych profilu twórcy
interface CreatorProfile {
  username: string;
  displayName: string;
  bio: string;
  avatarUrl: string;
  bannerUrl: string;
  goal?: { title: string; progress: number; target: number; }; // cel zbiórki
  tips: Tip[];
}

// Funkcja symulująca pobranie danych twórcy (placeholder zamiast faktycznego API)
async function fetchCreatorData(username: string): Promise<CreatorProfile> {
  // W prawdziwej aplikacji: wywołanie API, np. GET /api/creators/[username]
  return {
    username,
    displayName: "Jan Kowalski",
    bio: "Streamer gier retro. Kocham pixele i dobrą kawę!",
    avatarUrl: "/avatars/jan.png",
    bannerUrl: "/banners/retro-game.jpg",
    goal: { title: "Nowy mikrofon", progress: 40, target: 100 },
    tips: [
      { id: 1, from: "Mateusz", amount: 5, message: "Dzięki za stream!", date: "2025-07-01" },
      { id: 2, from: "Anonim", amount: 2, date: "2025-07-03" },
      // ...inne wpisy
    ]
  };
}

export default async function CreatorProfilePage({ params }: { params: { username: string } }) {
  const profile = await fetchCreatorData(params.username);
  const [isTipModalOpen, setTipModalOpen] = useState(false);

```

```

return (
  <div className="min-h-screen bg-brand-dark text-white">
    {/ * Banner + Avatar */}
    <div className="relative bg-gray-800">
      {/ * Banner image */}
      <Image src={profile.bannerUrl} alt="banner" width={1200} height={300}
        className="w-full h-48 object-cover opacity-80" />
      {/ * Avatar and name */}
      <div className="absolute inset-0 flex items-center justify-center md:justify-start
md:pl-10">
        <Image src={profile.avatarUrl} alt={profile.displayName} width={128} height={128}
          className="rounded-full border-4 border-brand-gold" />
        <div className="md:ml-6 mt-4 md:mt-0 text-center md:text-left">
          <h1 className="text-3xl font-bold">{profile.displayName}</h1>
          <p className="text-gray-200">{profile.bio}</p>
          {profile.goal && (
            <div className="mt-2">
              <div className="text-sm text-gray-300">{profile.goal.title} –
{profile.goal.progress}%</div>
              <div className="w-40 bg-gray-700 rounded-full h-2 mt-1">
                <div className="bg-brand-gold h-2 rounded-full" style={{ width:
`${profile.goal.progress}%` }} />
              </div>
            </div>
          )}
        </div>
      </div>
    </div>

    {/ * Main Content: Support button, alternative address, tip history */}
    <div className="max-w-3xl mx-auto px-4 py-6">
      {/ * Support Section */}
      <div className="flex items-center justify-between mb-6">
        {/ * Tip (Support) Button */}
        <TipButton onClick={() => setTipModalOpen(true)}>
          🍷 Wesprzyj
        </TipButton>
        {/ * Alternative wallet address icon */}
        <button
          onClick={() => {/ * TODO: open modal with QR code & address */}}
          className="flex items-center text-brand-gold hover:text-white"
          title="Adres portfela twórcy"
        >
          <QRCodeIcon className="w-6 h-6 mr-1" />
          <span className="hidden sm:inline">Adres do wpłaty</span>
        </button>
      </div>
    </div>

```

```

    { /* Tip History */
    <div className="bg-gray-800/50 p-4 rounded-lg">
      <h2 className="text-xl font-semibold mb-3">Ostatnie wsparcia</h2>
      {profile.tips.length > 0 ? (
        <TipHistory tips={profile.tips} />
      ) : (
        <p className="text-gray-400">Brak jeszcze napiwków. Bądź pierwszym
wspierającym!</p>
      )}
    </div>
  </div>

  { /* Tip Modal (Payment Flow) */
  {isTipModalOpen && (
    <TipModal creator={profile.username} onClose={() => setTipModalOpen(false)} />
  )}
  </div>
);
}

```

W powyższym kodzie warto zwrócić uwagę na następujące aspekty:

Zastosowano `getServerSideProps/fetchCreatorData` (tutaj uproszczony jako funkcja wywołana w komponencie `Async Server Component`) do pobrania danych profilu. W rzeczywistości `Next.js` może wyrenderować tę stronę statycznie (SSG) z danymi z bazy, ale dla celów demonstracyjnych dane są wypełnione statycznie.

Banner i Avatar: użyto komponentu `Image` `Next.js` do optymalnego wczytywania obrazków. Avatar jest nałożony na banner za pomocą `absolute` i stylowany złotą ramką (`border-brand-gold`) by wyraźnie się odcinał. Tekst na bannerze (nazwa, bio) jest półprzezroczysty lub jasny, by zachować czytelność na tle obrazka.

Goal (Cel zbiórki): jeżeli twórca ma aktywny cel, wyświetla się jego tytuł i procent realizacji, wraz z prostym paskiem postępu (ciemnoszare tło + złoty pasek wypełnienia o szerokości odpowiadającej procentowi).

Przycisk "Wesprzyj": wykorzystano komponent `TipButton` – może to być stylowany `<button>` korzystający z `Tailwind` (np. `bg-brand-gold text-black font-bold px-4 py-2 rounded`) albo z komponentu `UI` (`shadcn`). Ważne, by wyróżniał się na tle (tutaj czarny tekst na złotym tle, duży rozmiar). Po kliknięciu zmienia stan `isTipModalOpen` na `true`, co skutkuje wyświetleniem `<TipModal>`.

Ikona adresu portfela: pokazana jest opcjonalnie jako przycisk z ikoną QR. Po kliknięciu powinna otworzyć modal lub dropdown z informacją: adres portfela twórcy (pobierany z profilu, np. `profile.walletAddress`) i kod QR do zeskanowania. Ten modal nie jest w pełni zaimplementowany w kodzie (oznaczono `TODO`), ale należałoby go zrealizować analogicznie do `TipModal` – np. jako oddzielny komponent `AddressModal`.

Historia wsparcia: użyto komponentu TipHistory do wyświetlenia listy napiwków. Jego implementacja może wyglądać następująco (przykład poniżej). Jeśli lista tips jest pusta, wyświetlamy komunikat zachęcający do bycia pierwszym wspierającym. Historia buduje społeczny dowód – w stylu tablicy/komentarzy.

Przykładowa implementacja komponentu historii napiwków (TipHistory), używanego powyżej:

```
// components/TipHistory.tsx - lista ostatnich napiwków
import React from 'react';
import { User, Clock } from 'lucide-react'; // ikonki: user (dla anonimowych), clock (dla czasu)

interface Tip {
  id: number;
  from: string;
  amount: number;
  message?: string;
  date: string; // np. "2025-07-01"
}

export const TipHistory: React.FC<{ tips: Tip[] }> = ({ tips }) => {
  return (
    <ul className="space-y-3">
      {tips.map(tip => (
        <li key={tip.id} className="flex items-start bg-gray-900/50 p-3 rounded">
          {/* Icon or avatar of fan (if we had fan avatars) */}
          <div className="mr-3 mt-1 text-brand-gold">
            <User className="w-5 h-5" />
          </div>
          <div className="flex-1">
            <div className="text-sm text-gray-100">
              <span className="font-semibold">{tip.from || "Anonimowy"}:</span>{" "}
              <span>wsparł kwotą <b>${tip.amount.toFixed(2)}</b></span>
            </div>
            {tip.message && (
              <p className="text-gray-300 text-sm italic">{tip.message}</p>
            )}
          </div>
          <div className="text-xs text-gray-400 flex items-center">
            <Clock className="w-4 h-4 mr-1" />
            {new Date(tip.date).toLocaleDateString()}
          </div>
        </li>
      ))}
    </ul>
  );
};
```

Ten komponent iteruje przez listę napiwków i renderuje każdy w osobnym wierszu z informacją o nadawcy, kwocie, opcjonalnym komunikacie i dacie. Użyto ikonki użytkownika jako placeholder (można by rozbudować o avatary fanów, jeśli są dostępne). Styl zakłada półprzezroczyste tło (bg-gray-900/50) dla wpisu, co odróżnia go od tła strony, oraz drobne detale (kursywa dla wiadomości fana). Taki format listy jest przejrzysty i zachowuje spójny styl z resztą UI.

Nota UX: Profil twórcy został zaprojektowany tak, by maksymalnie ułatwić fanowi wsparcie finansowe przy jednoczesnym dostarczeniu kontekstu o twórcy. Dlatego duży złoty przycisk "Wesprzyj" jest od razu widoczny bez przewijania (zwłaszcza na desktop). Elementy dodatkowe, jak opis czy historia, są umieszczone poniżej – ważne, ale nie odciągające uwagi od CTA. Profil jest też zoptymalizowany pod mobile: avatar i nazwa centrowane, przycisk wsparcia ewentualnie stale widoczny na dole ekranu (można zastosować CSS sticky lub osobny dolny pasek na mobile). Dzięki opcji bezpośredniej wpłaty (adres wallet), platforma nie zamyka się na zaawansowanych użytkowników – mogą oni skorzystać z własnych narzędzi, co wpisuje się w filozofię Web3 (otwartość, brak lock-in).

Proces Wsparcia – Przepływ "Tip Flow" (Modal płatności)

Kluczowym elementem UX TipJar+ jest proces przekazywania napiwku. Został on zrealizowany jako modal (dialog) wyświetlany po kliknięciu "Wesprzyj". Ma to na celu zachowanie kontekstu – fan nie opuszcza strony twórcy, a jedynie wykonuje akcję na wierzchu niej. Cały proces został zaprojektowany tak, aby był maksymalnie uproszczony i zrozumiały nawet dla osób nieobeznanych z kryptowalutami.

Etapy interfejsu płatności (TipModal):

1. Wybór kwoty napiwku: Fan wskazuje kwotę, jaką chce przekazać twórcy. Minimalna kwota to np. \$1. Interfejs może oferować suwak do wyboru kwoty lub gotowe przyciski (np. \$1, \$5, \$10, \$50) oraz pole tekstowe do wpisania własnej kwoty. Kwota zawsze jest prezentowana w USDC (1 USDC ≈ \$1).

2. Wybór metody płatności: Platforma oferuje kilka dróg dokonania wpłaty:

Metoda wewnętrzna (TipJar Wallet): Jeśli fan jest zalogowany i posiada wewnętrzny portfel TipJar (custodial Circle Wallet), może przelać środki wewnętrznie. Wtedy kwota jest natychmiast transferowana w systemie (off-chain) z portfela fana na portfel twórcy. Ta metoda jest bezgazowa (rozliczenie wewn. w Circle).

Metoda zewnętrzna (Crypto Wallet): Fan może zapłacić ze swojego własnego portfela kryptowalutowego (np. MetaMask) – nawet bez zakładania konta na TipJar (gość). W takim przypadku wybiera on opcję np. "Zapłać przez Crypto Wallet", łączy portfel (pod spodem wykorzystujemy wagmi do integracji z MetaMask), a po wprowadzeniu kwoty transakcja jest realizowana on-chain. Dzięki integracji z Circle Gas Station / Paymaster opłata za gaz może być pokryta w USDC, więc fan nie musi posiadać ETH/MATIC – to nowoczesne rozwiązanie korzystające z account abstraction (jeden z trendów Web3 2024).

Metoda fiat (karta): (Planowane w przyszłości) Możliwość zapłaty kartą płatniczą lub np. Google Pay/Apple Pay – TipJar w tle przeliczy fiat na USDC i doda do portfela twórcy. W MVP można wyświetlić tę opcję jako wyszarzoną lub z adnotacją "wkrótce".

3. Finalizacja transakcji: Po wyborze kwoty i metody, fan zatwierdza płatność. W zależności od metody:

Dla portfela zewnętrznego – w przeglądarce pojawi się popup/metamask z prośbą o potwierdzenie transakcji on-chain (podpisanie).

Dla metody wewnętrznej – transakcja odbywa się od razu w systemie (wywołujemy endpoint backendu, który używa API Circle do transferu między portfelami).

Dla metody fiat – przekierowanie do bramki płatności lub obsługa poprzez SDK (w planach). Po pomyślnym przetworzeniu transakcji, modal wyświetla komunikat potwierdzenia, np. "Dziękujemy za wsparcie!". Lista napiwków na profilu twórcy powinna automatycznie zaktualizować się o nowy wpis (w idealnym przypadku w czasie rzeczywistym, np. przez WebSocket lub re-fetch danych). Twórca od razu widzi środki na swoim saldzie (wewn. portfel USDC).

4. Obsługa scenariuszy logowania: Jeśli fan nie jest zalogowany i wybierze metodę wymagającą konta (np. wewnętrzna), zostanie poproszony o zalogowanie/zarejestrowanie się przed finalizacją. Po logowaniu wróci z powrotem do modala z zachowaną kwotą/metodą. Jeśli pozostaje gościem, wciąż może użyć portfela zewnętrznego do zapłaty (bez konta).

5. Alternatywa – adres portfela: Jak wspomniano, fan może wybrać zamiast tego opcję "Adres do wpłaty" poza aplikacją. W praktyce jednak w modalu także dodajemy link "Masz własny portfel? Wyślij bezpośrednio na adres X" z wyświetleniem adresu/QR – co jest zbieżne z tą alternatywą, ale w ramach jednego UX.

Implementacja komponentu TipModal poniżej pokazuje uproszczony interfejs płatności. Wykorzystuje on bibliotekę HeadlessUI (Dialog) do stworzenia modala oraz wagmi do obsługi połączenia z portfelem (dla metody zewnętrznej). Stan formularza (kwota, wybór metody) jest zarządzany lokalnie za pomocą hooków React. W prawdziwej integracji wywołalibyśmy odpowiednie funkcje/API na submit.

```
// components/TipModal.tsx - Modal przekazywania napiwku  
"use client";
```

```
import { Dialog, Transition } from "@headlessui/react";
```

```

import { Fragment, useState } from "react";
import { useAccount, useConnect, useDisconnect } from "wagmi";
import { MetaMaskConnector } from 'wagmi/connectors/metaMask';

interface TipModalProps {
  creator: string;      // unikalny identyfikator twórcy (alias)
  onClose: () => void;
}

export const TipModal: React.FC<TipModalProps> = ({ creator, onClose }) => {
  const [amount, setAmount] = useState<number>(5); // domyślna kwota $5
  const [method, setMethod] = useState<"platform" | "wallet" | "card">("platform");
  const [loading, setLoading] = useState(false);
  const { connect, connectors } = useConnect({
    connectors: [new MetaMaskConnector()],
    chainId: 1 // przykładowo Ethereum mainnet lub sieć zgodna z USDC (np. Polygon)
  });
  const { disconnect } = useDisconnect();
  const { isConnected, address } = useAccount();

  // Wywoływane przy potwierdzeniu
  const handleSendTip = async () => {
    setLoading(true);
    try {
      if (method === "platform") {
        // Wywołaj API backendu do wewnętrznej transakcji (przekazanie z portfela fana na
        // twórcę)
        await fetch("/api/tips", {
          method: "POST",
          body: JSON.stringify({ to: creator, amount }),
          headers: { "Content-Type": "application/json" }
        });
      } else if (method === "wallet") {
        // Inicjuj transakcję on-chain przez portfel użytkownika (używając wagmi/viem)
        if (!isConnected) {
          // Jeśli nie podłączono portfela, łącz automatycznie (MetaMask)
          await connect({ connector: connectors[0] });
        }
        // Zakładamy że połączono i mamy address
        // Wywołanie inteligentnego kontraktu lub transferu USDC on-chain przez viem
        // (pomijamy szczegóły)
        console.log(`Sending ${amount} USDC on-chain from ${address} to ${creator}'s
        address...`);
      } else if (method === "card") {
        // Placeholder: integracja z bramką fiat (np. wywołanie endpointu utworzenia płatności
        // Circle)
        window.alert("Integracja płatności kartą w przygotowaniu.");
      }
      // Po pomyślnej transakcji zamykamy modal
    } catch {
      // ...
    }
  };

```



```

    onClose();
  } catch (err) {
    console.error("Payment error", err);
    // TODO: obsługa błędów (np. komunikat o błędzie)
  } finally {
    setLoading(false);
  }
};

return (
  <Transition show as={Fragment}>
    <Dialog as="div" className="relative z-50" onClose={onClose}>
      {/* Overlay */}
      <Transition.Child as={Fragment}
        enter="ease-out duration-300" enterFrom="opacity-0" enterTo="opacity-100"
        leave="ease-in duration-200" leaveFrom="opacity-100" leaveTo="opacity-0">
        <div className="fixed inset-0 bg-black/60" aria-hidden="true" />
        </Transition.Child>

        {/* Modal panel */}
        <Transition.Child as={Fragment}
          enter="ease-out duration-300" enterFrom="opacity-0 translate-y-4"
          enterTo="opacity-100 translate-y-0"
          leave="ease-in duration-200" leaveFrom="opacity-100 translate-y-0"
          leaveTo="opacity-0 -translate-y-4">
          <div className="fixed inset-0 flex items-center justify-center">
            <Dialog.Panel className="mx-4 max-w-md w-full bg-brand-dark rounded-lg p-6
            border border-brand-gold">
              <Dialog.Title className="text-xl font-bold text-brand-gold mb-4">
                Wsparcie twórcy @{creator}
              </Dialog.Title>

              {/* Kwota napiwku */}
              <label className="block mb-3">
                <span className="text-gray-200">Kwota napiwku (USDC):</span>
                <input
                  type="number" min={1} step={1} value={amount}
                  onChange={(e) => setAmount(Number(e.target.value))}
                  className="mt-1 block w-full bg-gray-800 border border-gray-600 rounded px-3
py-2 text-white"
                />
              </label>

              {/* Przykładowe szybkie przyciski kwot */}
              <div className="flex space-x-2 mb-4">
                {[1,5,10,50].map(val => (
                  <button key={val}
                    className={`px-3 py-1 rounded ${amount===val ? 'bg-brand-gold text-black
font-bold' : 'bg-gray-700 text-gray-200`}

```

```

        onClick={() => setAmount(val)}>
        ${val}
    </button>
  )}
</div>

{/* Wybór metody płatności */}
<div className="mb-4">
  <span className="text-gray-200">Metoda płatności:</span>
  <div className="mt-2 space-y-2">
    <label className="flex items-center">
      <input type="radio" name="method" value="platform"
        checked={method==="platform"}
        onChange={() => setMethod("platform")}
        className="form-radio text-brand-gold" />
      <span className="ml-2 text-white">Portfel TipJar</span>
    </label>
    <label className="flex items-center">
      <input type="radio" name="method" value="wallet"
        checked={method==="wallet"}
        onChange={() => setMethod("wallet")}
        className="form-radio text-brand-gold" />
      <span className="ml-2 text-white">Mój portfel kryptowalutowy</span>
      {!isConnected ? (
        <button onClick={() => connect({ connector: connectors[0] })}
          className="ml-3 px-2 py-1 text-xs border border-brand-gold
text-brand-gold rounded">
            Połącz portfel
          </button>
        ) : (
          <span className="ml-2 text-xs text-gray-400">(podłączono:
{address?.slice(0,6)}...)</span>
        )}
      </label>
    <label className="flex items-center">
      <input type="radio" name="method" value="card"
        checked={method==="card"}
        onChange={() => setMethod("card")}
        className="form-radio text-brand-gold" />
      <span className="ml-2 text-white">Karta płatnicza (fiat)</span>
      <span className="ml-2 text-xs text-gray-500">wkrótce</span>
    </label>
  </div>
</div>

{/* Przyciski akcji: Wyślij napiwek lub Anuluj */}
<div className="mt-6 flex justify-end space-x-3">
  <button

```

```

        className="px-4 py-2 rounded bg-gray-600 text-white"
        onClick={onClose}
        disabled={loading}
      >
        Anuluj
      </button>
      <button
        className="px-4 py-2 rounded bg-brand-gold text-black font-semibold
disabled:opacity-50"
        onClick={handleSendTip}
        disabled={loading}
      >
        {loading ? "Wysyłanie..." : "Wyślij napiwek"}
      </button>
    </div>
  </Dialog.Panel>
</div>
</Transition.Child>
</Dialog>
</Transition>
);
};

```

Omówienie TipModal: Użyto komponentów <Dialog> z Headless UI, które zapewniają szkielet dostępnego modala (focus trap, esc-to-close, itp.). Pojawianie się i znikanie modala jest animowane za pomocą <Transition> z klasami Tailwind (fade i slide). Modal sam w sobie ma ciemne tło i złotą ramkę (border-brand-gold), aby wyróżnić go wizualnie. Wewnątrz:

Pole kwoty: standardowy <input type="number"> z minimalną wartością 1. Do tego kilka przycisków szybkiego wyboru kwoty – zmieniają one stan amount.

Wybór metody płatności: zrealizowany jako radio buttony. Dla każdej opcji:

"Portfel TipJar" (platform) – dla zalogowanych. Jeśli użytkownik nie jest zalogowany, przy próbie wysłania należałoby przechwycić to i skierować do logowania (tu zakładamy, że jeśli wybrał tę opcję i nie ma konta, to UI już powinno go ostrzec lub zablokować).

"Mój portfel" (wallet) – integracja z wagmi. Jeśli portfel nie jest połączony, wyświetla się przycisk "Połącz portfel" (który wywołuje connect z MetaMaskConnector). Jeśli jest połączony, pokazujemy krótki fragment adresu jako potwierdzenie. (W prawdziwej aplikacji należałoby obsłużyć sieć – np. upewnić się, że user jest na właściwej sieci zgodnej z TipJar, oraz ewentualnie zmienić chain).

"Karta płatnicza" (card) – tutaj oznaczona jako "wkrótce". Jest wybrana radio, ale w momencie próby wysłania wyświetli tylko alert, że opcja nieaktywna.

Logika `handleSendTip`: W zależności od metody, podejmujemy odpowiednie akcje. Dla metody platformy – wywołujemy endpoint API (np. POST na `/api/tips`) – w praktyce backend sprawdzi saldo fana i zainicjuje transfer w Circle (moduł Payments). Dla metody wallet – jeśli portfel nie był połączony, najpierw go łączymy, potem (to jest bardzo uproszczone) należałoby wywołać funkcję transferu USDC on-chain. W tym celu można skorzystać z biblioteki `viem` lub `wagmi`: np. przygotować transakcję ERC-20 transferu z adresu fana (connected wallet) na adres twórcy. Ponieważ TipJar planuje używać `gas paymaster`, transakcja mogłaby być podpisana w specjalny sposób – to jednak wykracza poza zakres frontendu (backend może przygotować tzw. meta-transakcję). Tu jest to zilustrowane jedynie logiem w konsoli. Dla karty – alert (brak implementacji).

Po pomyślnej operacji `onClose()` jest wywoływane, co zamyka modal. Ewentualnie można pokusić się o bardziej rozbudowane UX: np. wyświetlenie komunikatu sukcesu przed zamknięciem.

Modal jest w trybie `show` bazującym na zewnętrznym stanie (w komponentach wyżej `isTipModalOpen`). Zwróćmy uwagę, że TipModal jest oznaczony `"use client"` – bo korzysta ze stanów i `wagmi` (które działa po stronie klienta). W Next.js App Router, `CreatorProfilePage` może być komponentem `server` (co ułatwia pobranie danych), a TipModal i inne interaktywne elementy mogą być komponentami klienta wstrzykniętymi wewnątrz.

Nota UX: Interfejs płatności został zaprojektowany tak, by uprościć użytkownikowi podjęcie decyzji. Domyślna kwota \$5 jest zaznaczona (sugerowana, ale łatwo zmienić). UI stara się mówić językiem zrozumiałym dla laika – np. "Portfel TipJar" zamiast technicznego "transakcja wewnętrzna off-chain", oraz "Mój portfel kryptowalutowy" zamiast np. "użyj MetaMask". Dzięki temu użytkownik mniej obeznany wybierze metodę platformową (jeśli jest zalogowany) lub zostanie do tego zachęcony, natomiast zaawansowany użytkownik od razu rozpozna opcję portfela i z niej skorzysta. Ważnym elementem jest wyjaśnienie, że nie trzeba posiadać ETH na opłaty – to można dodać np. jako tooltip lub krótki tekst przy opcji portfela: "Dzięki integracji TipJar opłata za gas zostanie pobrana w USDC". Modal pokrywa cały ekran półprzezroczystym tłem, co skupia uwagę użytkownika na wykonaniu akcji. W przypadku błędu (np. brak środków fana, błąd transakcji) – należałoby wyświetlić komunikat o błędzie w modalu (np. czerwony tekst). Przy zamknięciu modala (Anuluj) żadne zmiany nie są wykonywane.

Rejestracja / Logowanie

Ekran rejestracji i logowania umożliwiają utworzenie konta twórcy lub fana oraz dostęp do platformy. Zgodnie z projektem TipJar+, proces ten powinien być jak najprostszy i najszybszy, z preferencją dla metod logowania zewnętrznego (OAuth), co redukuje tarcie. Równocześnie, dla zaawansowanych użytkowników dodano opcję Web3 Sign-In (SIWE), a dla tradycyjistów – klasyczne hasło.

Opcje rejestracji/logowania:

Logowanie przez OAuth: Najłatwiejsza metoda – przyciski typu "Kontynuuj przez Google", "Kontynuuj przez Twitch" itp. Po kliknięciu użytkownik przechodzi standardowy OAuth flow, a

po powrocie jest automatycznie zalogowany. W UI realizujemy to jako wyróżnione przyciski z logo danej usługi.

Rejestracja przez e-mail/hasło: Tradycyjna metoda – formularz z polami e-mail, hasło (i potwierdzenie hasła jeśli rejestracja) oraz przycisk "Zarejestruj się". Po rejestracji należy poinformować o konieczności weryfikacji e-mail (link aktywacyjny), zwłaszcza dla twórców.

SIWE (Sign-In with Ethereum): Dla osób posiadających portfel web3 – możliwość zalogowania poprzez podpisanie wiadomości swoim portfelem (np. MetaMask). W praktyce UI wyświetla przycisk "Zaloguj przez Ethereum" – po kliknięciu łączy portfel (wagmi) i wywołuje backend (np. endpoint /auth/siwe do pobrania nonce, następnie podpis i weryfikacja). Jeżeli użytkownik ma już konto powiązane z tym adresem, zostanie zalogowany, jeśli nie – można utworzyć nowe (np. twórcy posiadający portfel mogą nie chcieć przechodzić przez OAuth).

Wybór roli (przy rejestracji): Ponieważ TipJar+ obsługuje twórców i fanów, podczas rejestracji twórcy przechodzą dodatkowy krok konfiguracji (wybór aliasu, generowanie portfela). W UI możemy mieć oddzielne ścieżki: np. osobny przycisk "Założ konto twórcy" vs "Zarejestruj się jako fan". Alternatywnie, pojedynczy formularz rejestracji może zawierać pytanie o typ konta lub algorytm: rejestracja ze strony głównej "Założ profil twórcy" domyślnie ustawia typ konta = CREATOR.

Flow powrotu przy płatności: Jak wspomniano, jeśli niezalogowany fan trafił do rejestracji w trakcie procesu wsparcia (kliknął Wesprzyj → zaloguj), to po zalogowaniu/rejestracji powinien zostać zawrócony do kontekstu wysyłania napiwku. Realizuje się to zwykle przez mechanizm przekazywania URL powrotu (np. param w query lub w stanie aplikacji).

Poniżej przykład prostego ekranu rejestracji (app/auth/register/page.tsx). Dla zwięzłości, łączymy w nim możliwość przełączenia na "Zaloguj się" – realnie można to rozdzielić na oddzielne sub-routes lub komponenty. Wykorzystujemy komponenty formularza z Tailwind (lub @tailwindcss/forms), a także integrujemy przyciski OAuth oraz wallet connect (SIWE) za pomocą wagmi.

```
// app/auth/register/page.tsx - Rejestracja / Logowanie
"use client";
```

```
import { useState } from 'react';
import { useConnect, useAccount } from 'wagmi';
import { MetaMaskConnector } from 'wagmi/connectors/metaMask';

export default function RegisterPage() {
  const [isRegister, setIsRegister] = useState(true);
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const { connect } = useConnect({ connectors: [new MetaMaskConnector()] });
  const { address } = useAccount();
```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  // Wywołanie endpointu rejestracji/logowania
  const url = isRegister ? "/api/auth/register" : "/api/auth/login";
  const res = await fetch(url, {
    method: "POST",
    body: JSON.stringify({ email, password }),
    headers: { "Content-Type": "application/json" }
  });
  if (res.ok) {
    // Sukces: w praktyce ustawienie stanu user (np. w zustand) i przekierowanie
    window.location.href = "/dashboard";
  } else {
    alert("Błąd podczas uwierzytelniania. Upewnij się, że dane są poprawne.");
  }
};

return (
  <div className="min-h-screen bg-brand-dark text-white flex items-center justify-center p-4">
    <div className="max-w-md w-full bg-gray-800 p-6 rounded-lg border border-gray-700">
      <h1 className="text-2xl font-bold mb-6">{isRegister ? "Zarejestruj się" : "Zaloguj się"}</h1>

      {/* OAuth buttons */}
      <div className="space-y-3 mb-6">
        <button className="w-full bg-white text-black font-medium py-2 px-4 rounded flex items-center justify-center"
          onClick={() => { window.location.href = "/api/auth/google"; }}>
           Kontynuuj przez Google
        </button>
        <button className="w-full bg-[#9146FF] text-white font-medium py-2 px-4 rounded flex items-center justify-center"
          onClick={() => { window.location.href = "/api/auth/twitch"; }}>
           Kontynuuj przez Twitch
        </button>
      </div>

      {/* Divider */}
      <div className="text-center text-gray-400 mb-6">lub {isRegister ? "utwórz konto:" : "zaloguj się:"}</div>

      {/* Email/Password form */}
      <form onSubmit={handleSubmit} className="space-y-4">
        <div>
          <label className="block text-sm mb-1">Email:</label>

```

```

        <input type="email" value={email} onChange={e => setEmail(e.target.value)}
          className="w-full px-3 py-2 bg-gray-700 text-white rounded border
border-gray-600" required />
      </div>
    </div>
    <label className="block text-sm mb-1">Hasło:</label>
    <input type="password" value={password} onChange={e =>
setPassword(e.target.value)}
      className="w-full px-3 py-2 bg-gray-700 text-white rounded border
border-gray-600" required />
    </div>
    <button type="submit" className="w-full bg-brand-gold text-black font-semibold py-2
px-4 rounded">
      {isRegister ? "Zarejestruj się" : "Zaloguj się"}
    </button>
  </form>

  {/* SIWE login */}
  <div className="mt-6 text-center">
    <button onClick={() => connect()} className="text-brand-gold underline">
      {address ? "Połączono: " + address.slice(0,6) + "..." : "Zaloguj przez Ethereum
(Web3)"}
    </button>
  </div>

  {/* Toggle login/register */}
  <p className="mt-4 text-sm text-center text-gray-300">
    {isRegister ? "Masz już konto?" : "Nie masz konta?"}{""}
    <a href="#" onClick={() => setIsRegister(!isRegister)} className="text-brand-gold
underline">
      {isRegister ? "Zaloguj się" : "Zarejestruj się"}
    </a>
  </p>
</div>
</div>
);
}

```

Objaśnienia: Ten komponent pokazuje podstawowy układ formularza. Na górze dwie duże akcje OAuth (Google i Twitch) – stylowane w kolorach natywnych (biały dla Google z czarnym tekstem i logo, fiolet Twitcha z białym tekstem). Niżej klasyczny formularz email/hasło, i na dole opcja SIWE: przycisk, który korzysta z wagmi do podłączenia portfela. Po połączeniu portfela, w address będzie dostępny adres – tu prosto pokazujemy kawałek adresu. W realnej implementacji, kliknięcie tego powinno zainicjować proces SIWE: backend endpoint do wygenerowania nonce, potem podpis przez wagmi (signMessage), i weryfikacja na backendzie. Ze względu na złożoność, nie pokazano tu całego procesu, ale miejsce jest wskazane. Po udanym logowaniu/rejestracji wykonujemy przekierowanie do /dashboard (lub powrót do redirect jeśli był ustawiony).

Po rejestracji twórcy: Zgodnie z poradnikiem, twórca po pierwszym logowaniu przechodzi jednorazowo przez konfigurator profilu (wybór aliasu, informacje o portfelu Circle, uzupełnienie profilu). W powyższym kodzie nie rozwinięto tego – zakładamy, że backend tworzy portfel i alias, a UI może przekierować np. do /dashboard/profile-setup jeśli potrzebny dodatkowy krok. W MVP można uprościć: alias twórcy = część email przed @ lub unikalny ID, a profil edytuje potem w ustawieniach.

Nota UX: Ekran logowania jest minimalistyczny i skupia się na szybkim onboardingu użytkownika. Najczęściej wybierane opcje (Google) są na górze i wyraźne. Kolorystyka pozostaje w dark mode, ale np. użycie białego tła dla przycisku Google jest celowe – użytkownicy kojarzą tę konwencję i od razu widzą opcję. Pola formularza mają czytelne etykiety i duże pola kliknięcia. Całość jest wyśrodkowana i ma ograniczoną szerokość, by użytkownik nie czuł się przytłoczony. Wiadomość o błędzie (tu uproszczona jako alert) w realnym produkcie powinna pojawić się np. jako czerwony baner/pasek w formularzu. Dzięki integracji z różnymi metodami logowania, bariera wejścia jest niska – nawet jeśli ktoś nie ma konta społecznościowego, może użyć e-mail, a entuzjaści krypto mogą od razu skorzystać z portfela.

Panel Twórcy (Dashboard)

Po zalogowaniu twórca ma dostęp do swojego panelu kontrolnego, gdzie może zarządzać profilem, przeglądać statystyki i wykonywać operacje (np. wypłaty). Panel ten został zaprojektowany jako aplikacja jednostronicowa wewnątrz frontendu – tzn. przełączanie podstron odbywa się płynnie, bez pełnych przeładowań, dla lepszego UX. W Next.js wykorzystujemy do tego wspólny layout i nawigację kliencką.

Panel twórcy składa się z kilku głównych sekcji (zgodnie z dokumentacją):

Statystyki Napiwków (Support Overview): Strona główna dashboardu (po zalogowaniu) prezentująca podsumowanie aktywności finansowej twórcy. Zawiera:

Łączna kwota wsparcia (sumarycznie w USDC) i liczba wspierających.

Wykres historyczny (np. słupkowy z sumami tygodniowymi lub liniowy dzienny) ilustrujący otrzymane napiwki w czasie.

Lista ostatnich napiwków (podobna do tej na publicznym profilu, może być bardziej rozbudowana o filtr “tylko od ostatniego logowania”).

Top fani (opcjonalnie) – ranking fanów wg sumy wsparcia.

CTA do udostępnienia profilu (np. przycisk generujący link lub kod QR do profilu twórcy, zachęcający do promocji).

Ustawienia Strony (Profile Customization): Sekcja pozwalająca edytować publiczny profil twórcy. Twórca może zmienić:

Avatar i baner (upload obrazków),

Wyświetlaną nazwę (oraz ewentualnie alias, choć to może być ograniczone po rejestracji),

Opis (bio),

Ustawić cel zbiórki (kwota docelowa, opis celu),

Dodać linki do social mediów (Twitch, YouTube, Twitter itd.), które będą się wyświetlać jako ikonki na profilu.

Widzieć podgląd na żywo swojego profilu podczas edycji (np. obok formularza).

Opcje Wypłat (Withdrawal Options): Miejsce do zarządzania zgromadzonymi środkami i wypłatami. Twórca widzi:

Swoje bieżące saldo USDC (odczyt z API Circle),

Może zdefiniować adres wypłaty – np. adres własnego portfela (EOA) Ethereum/Polygon. Jeśli twórca wcześniej połączył swój portfel (przez SIWE), ten adres może być automatycznie wypełniony jako zweryfikowany.

Formularz zlecenia wypłaty: pole kwoty do wypłaty (z walidacją do salda) i przycisk "Wypłać" (oraz ewentualnie "Wypłać wszystko"). Po zatwierdzeniu, środki są wysyłane z portfela Circle twórcy na wskazany adres (minus prowizja).

Historię wypłat: lista wykonanych wypłat z datami, kwotami, statusem (np. Completed/Pending).

Informację o przybliżonym czasie wypłaty (np. „Środki powinny dotrzeć w ciągu kilku minut” jeśli on-chain, albo „1-2 dni robocze” jeśli fiat).

Cel (Goal) i Subskrypcje: W MVP cele zbiórek są zarządzane w ustawieniach profilu, nie jako osobna sekcja. Subskrypcje (cykliczne wsparcie) to funkcja planowana – panel jest zaprojektowany tak, by można ją dodać (np. kolejna zakładka). W MVP jednak możemy ją pominąć lub dodać placeholder „wkrótce”.

Inne: Potencjalnie panel może zawierać sekcję powiadomień, ustawień konta (zmiana hasła, 2FA), ale to wykracza poza podstawowy zakres MVP.

Struktura UI panelu: Panel składa się z nawigacji bocznej (Sidebar), nagłówka oraz głównej przestrzeni treści.

Sidebar: Zawiera menu nawigacyjne pomiędzy podstronami panelu. Ikony+etykiety np.: Dashboard (Statystyki), Profile Setup, Withdrawals, (ew. Subscriptions). Na mobile jest ukrywany – można zrobić go wysuwającym drawerem.

Nagłówek (Header Bar): Górny pasek, pokazujący nazwę aplikacji/logo, tytuł aktualnej sekcji, oraz ewentualnie avatara twórcy z menu rozwijanym (np. z opcjami „Idź do profilu publicznego”, „Wyloguj”). Tu też umieszczamy przycisk przełącznika trybu jasny/ciemny.

Main Content: Wydzielona przestrzeń, gdzie wyświetlane są poszczególne widoki sekcji. Dzięki Next.js layout, przy nawigacji zmienia się tylko ta część, co jest szybkie i może być animowane.

Poniżej przykład implementacji layoutu i jednej z sekcji panelu – Statystyki Napiwków (app/dashboard/stats/page.tsx). Zakładamy, że layout app/dashboard/layout.tsx zapewnia wspólny układ z Sidebar i Header.

```
// app/dashboard/layout.tsx - Layout dla panelu twórcy (Dashboard)
import { DashboardSidebar } from '@components/DashboardSidebar';
import { DashboardHeader } from '@components/DashboardHeader';

export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="min-h-screen flex bg-brand-dark text-white">
      {/* Sidebar */}
      <DashboardSidebar />
      <div className="flex-1 flex flex-col">
        {/* Header bar */}
        <DashboardHeader />
        {/* Main content area */}
        <main className="flex-1 p-6">
          {children}
        </main>
      </div>
    </div>
  );
}
```

Kod layoutu jest prosty: dzieli ekran na boczny sidebar (stałe widoczny na desktop) i główny obszar zawierający header na górze i zmieniającą się zawartość poniżej. Przykładowy sidebar i header:

```
// components/DashboardSidebar.tsx
import { PieChart, Settings, Wallet, LogOut } from 'lucide-react';
import Link from 'next/link';

export function DashboardSidebar() {
  return (
```

```

<nav className="w-64 bg-gray-900 flex flex-col p-4 space-y-2">
  <h2 className="text-xl font-bold mb-4">TipJar+ Panel</h2>
  <Link href="/dashboard/stats" className="flex items-center p-2 rounded
hover:bg-gray-800">
    <PieChart className="w-5 h-5 mr-2 text-brand-gold" /> Statystyki
  </Link>
  <Link href="/dashboard/profile" className="flex items-center p-2 rounded
hover:bg-gray-800">
    <Settings className="w-5 h-5 mr-2 text-brand-gold" /> Ustawienia Strony
  </Link>
  <Link href="/dashboard/withdrawals" className="flex items-center p-2 rounded
hover:bg-gray-800">
    <Wallet className="w-5 h-5 mr-2 text-brand-gold" /> Wyплаты
  </Link>
  {/* Można dodać Subskrypcje, itp. */}
  <div className="mt-auto pt-4 border-t border-gray-800">
    <button onClick={() => {/* Wyloguj (np. usuń token + redirect) */}}
      className="flex items-center w-full p-2 rounded hover:bg-gray-800 text-red-400">
      <LogOut className="w-5 h-5 mr-2" /> Wyloguj
    </button>
  </div>
</nav>
);
}

```

// components/DashboardHeader.tsx

import { Moon, Sun } from 'lucide-react';

import { useStore } from '@store/useStore'; // przykładowy store Zustand dla motywu

```

export function DashboardHeader() {
  const { theme, toggleTheme } = useStore(); // zakładamy, że store ma theme: 'dark' | 'light'
  return (
    <header className="bg-gray-900 flex items-center justify-between px-6 py-4 border-b
border-gray-800">
      <div className="text-xl font-semibold">Panel Twórcy</div>
      <div className="flex items-center space-x-4">
        {/* Switch theme */}
        <button onClick={toggleTheme} className="p-2 rounded hover:bg-gray-800">
          {theme === 'dark' ? <Sun className="w-5 h-5 text-white" /> : <Moon
className="w-5 h-5 text-yellow-500" />}
        </button>
        {/* Avatar/user menu placeholder */}
        <div className="relative">
          <button className="flex items-center focus:outline-none">
            
            <span>Moje konto</span>
          </button>
          {/* dropdown menu could appear here on click */}

```

```

    </div>
  </div>
</header>
);
}

```

Wyjaśnienia: Sidebar zawiera linki (Next.js Link) do poszczególnych podstron panelu. Użyto ikon z lucide (PieChart, Settings, Wallet, LogOut) z kolorami brandowymi dla akcentów. Aktualnie aktywny link można by wyróżnić np. innym tłem lub kolorem. Na dole sidebaru dodano przycisk Wyloguj – usuwa token sesji (np. czyści cookie/stan) i przekierowuje na stronę główną lub logowania.

Header pokazuje tytuł bieżącej sekcji (tu statycznie "Panel Twórcy" – można dynamicznie zmieniać np. na nazwę sekcji wykorzystując mechanizmy metadata w Next 13). Po prawej jest przełącznik trybu (ikona słońce/księżyc). Wykorzystujemy useStore (zustand) do pobrania aktualnego motywu i funkcji toggle. Ten store mógłby być zdefiniowany następująco:

```

// store/useStore.ts - przykładowy store Zustand
import create from 'zustand';

interface AppState {
  theme: 'dark' | 'light';
  toggleTheme: () => void;
  user?: { name: string; email: string; /* etc */ };
  setUser: (user: AppState['user']) => void;
}

export const useStore = create<AppState>((set) => ({
  theme: 'dark',
  toggleTheme: () => set((state) => {
    const newTheme = state.theme === 'dark' ? 'light' : 'dark';
    // dodaj/usuń klasę na <html> dla Tailwind
    if (typeof document !== 'undefined') {
      document.documentElement.classList.toggle('dark', newTheme === 'dark');
    }
    return { theme: newTheme };
  }),
  user: undefined,
  setUser: (user) => set({ user })
}));

```

Ten store inicjalizuje theme na 'dark' i definiuje toggleTheme (który też zarządza klasą CSS na dokumencie). Przechowuje też obiekt user z danymi zalogowanego użytkownika (tu uproszczony; zazwyczaj zawiera token lub przynajmniej ID). Po zalogowaniu setUser byłoby wywołane z danymi z API.

Teraz przykładowa sekcja Statystyki – zawiera wykres i listę napiwków. Wykorzystamy komponent TipHistory ponownie oraz założymy, że jest komponent StatsChart (np. oparty o Chart.js lub d3 – nie zagłębiamy implementacji wykresu). Pokażemy jak może wyglądać strona statystyk:

```
// app/dashboard/stats/page.tsx - Statystyki Napiwków
"use client";
```

```
import { TipHistory } from '@components/TipHistory';
import { useEffect, useState } from 'react';
// Placeholder for chart data
interface Tip { amount: number; date: string; }
```

```
export default function StatsPage() {
  const [tips, setTips] = useState<Tip[]>([]);
  const [total, setTotal] = useState(0);
  const [supporters, setSupporters] = useState(0);
```

```
  useEffect(() => {
    // TODO: fetch stats from API, here we simulate:
    const fakeTips: Tip[] = [
      { amount: 5, date: "2025-07-10" },
      { amount: 2, date: "2025-07-11" },
      { amount: 10, date: "2025-07-12" },
      // ... więcej danych
    ];
    setTips(fakeTips);
    setTotal(fakeTips.reduce((sum, t) => sum + t.amount, 0));
    setSupporters(50); // założymy 50 unikalnych wspierających
  }, []);
```

```
  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-bold">Twoje wsparcie</h1>
      { /* Summary cards */ }
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        <div className="bg-gray-800 p-4 rounded">
          <div className="text-sm text-gray-400">Łączna kwota wsparcia</div>
          <div className="text-2xl font-semibold text-brand-gold">${total.toFixed(2)}</div>
        </div>
        <div className="bg-gray-800 p-4 rounded">
          <div className="text-sm text-gray-400">Łączna liczba wspierających</div>
          <div className="text-2xl font-semibold text-brand-gold">{supporters}</div>
        </div>
        <div className="bg-gray-800 p-4 rounded">
          <div className="text-sm text-gray-400">Średnio na dzień</div>
          <div className="text-2xl font-semibold text-brand-gold">${(total/30).toFixed(2)}</div>
        </div>
      </div>
    </div>
```

```

    </div>
    { /* Chart (placeholder, e.g., use Chart.js) */ }
    <div className="bg-gray-800 p-4 rounded">
      <div className="text-sm text-gray-400 mb-2">Historia napiwków (ostatnie 30
dni)</div>
      <div className="h-40 bg-gray-700 rounded"></div> { /* tutaj byłby wykres */ }
    </div>
    { /* Recent tips list */ }
    <div className="bg-gray-800 p-4 rounded">
      <h2 className="text-lg font-semibold mb-3">Ostatnie napiwki</h2>
      { /* Reuse TipHistory (needs adapting to accept maybe top N) */ }
      <TipHistory tips={tips.map((t, idx) => ({
        id: idx, from: "Fan " + idx, amount: t.amount, date: t.date
      })))} />
    </div>
    { /* CTA to share profile */ }
    <div className="bg-brand-gold/10 border border-brand-gold p-4 rounded text-center">
      <p className="text-sm text-gray-100 mb-2">
        Pokaż światu swój profil TipJar+ i zdobądź więcej wsparcia!
      </p>
      <button className="bg-brand-gold text-black font-medium px-4 py-2 rounded">
        Udostępnij profil
      </button>
    </div>
  </div>
);
}

```

Ta strona pokazuje wykorzystanie danych statystycznych. Na górze szybki przegląd (kwota, liczba wspierających, średnia dzienna). Pod spodem sekcja wykresu (tu tylko placeholder – faktycznie można wykorzystać np. `react-chartjs-2` do wpięcia wykresu liniowego), a dalej lista ostatnich napiwków (tu używamy ponownie `TipHistory`). Na końcu jest CTA zachęcające twórcę do udostępnienia swojego profilu – można to powiązać z funkcją generującą link lub integracją z social mediami (np. tweet "Wesprzyj mnie na TipJar+"). Wszystko jest utrzymane w stylistyce dark mode.

Animacje w dashboardzie: Przejścia między poszczególnymi sekcjami panelu są płynne – ponieważ `Next.js App Router` nie przeladowuje całej strony, zmienia się tylko zawartość `<main>`. Można dodatkowo użyć `framer-motion` do animowania wejścia komponentów – np. opakować dzieci w `<AnimatePresence>` i `<motion.div>` z efektami `fade/slide`. Dodatkowo wprowadzenie nowych elementów, np. powiadomienia o nowym napiwku, może mieć krótką animację podkreślającą (w dokumentacji sugerowano miganie złotym tłem) – to detal do implementacji np. przez dodanie klasy CSS z animacją.

Responsywność panelu: Na węższych ekranach sidebar może być domyślnie ukryty – można go zrobić jako wysuwany panel (np. hamburger menu w headerze otwiera sidebar). W kodzie dla uproszczenia sidebar jest na stałe (co przy `w-64` i `overflow scroll` w main, spowoduje poziomy przewijany layout na mobilkach, co należałoby poprawić). W praktyce

wykorzysta się np. układ z Drawer z HeadlessUI lub po prostu warunkowe wstawienie sidebaru w DOM.

Podsumowanie UX panelu: Panel twórcy stara się dostarczyć konkretne informacje i narzędzia bez przeładowania interfejsu. Rozdzielenie na zakładki/sekcje powoduje, że użytkownik skupia się na jednym zadaniu na raz. Stylistyka jest spójna z resztą aplikacji: ciemne tła, złote nagłówki/ikony, czytelne białe teksty. Dzięki animacjom i jednopodstroncowemu charakterowi panel wydaje się szybki i nowoczesny, co buduje pozytywne doświadczenie (ważne, bo twórcy będą spędzać w nim dużo czasu). Zwrócono uwagę na szczegóły takie jak lazy loading obrazów (Next Image w awatarze, banerze) i optymalizacja wydajności, żeby panel działał płynnie.

Dokumentacja wdrożeniowa (README)

Poniżej zwięzła instrukcja dla deweloperów, jak uruchomić i integrować front-end TipJar+ z resztą systemu:

1. Wymagania wstępne: Upewnij się, że masz zainstalowane Node.js (≥ 18) i npm/yarn. Projekt jest utworzony w Next.js – zalecana jest znajomość Next 13 (App Router) oraz posiadanie globalnie zainstalowanego pnpm lub yarn jeśli preferowane.

2. Instalacja zależności: Sklonuj repozytorium frontendu, następnie w folderze projektu uruchom `npm install` (zainstaluje m.in. Tailwind, headlessui, lucide, framer-motion, wagmi, shadcn/ui komponenty itp.). Wszystkie wymagane pakiety są wymienione w `package.json`.

3. Konfiguracja środowiska: Utwórz plik `.env.local` z odpowiednimi zmiennymi:

`NEXT_PUBLIC_API_URL` – URL do backend API (NestJS), jeśli potrzebne dla wywołań z przeglądarki.

Klucze lub ID aplikacji OAuth (Google, Twitch) mogą być wymagane po stronie frontendu (np. do NextAuth lub widgetów) – skonfiguruj je w `.env` lub bezpośrednio w panelu dev usług (redirect URI to domena projektu + `/api/auth/callback/...`).

(Opcjonalnie) Konfiguracja wagmi: w pliku `lib/wagmi.ts` można ustawić sieć (chain) i klucze RPC. Domyślnie projekt może korzystać z publicznych RPC lub z konfiguracji wagmi domyślnej.

4. Uruchomienie dev: Wykonaj `npm run dev`. Aplikacja powinna wystartować na `http://localhost:3000`. Strona główna powinna wyświetlić landing z sekcjami jak opisane powyżej. Możesz przetestować responsywność zmniejszając okno przeglądarki – elementy powinny się układać kolumnowo na mobile.

5. Integracja z backendem: W miejscach oznaczonych komentarzem TODO lub obecnie wywołujących fikcyjne API, podłącz prawdziwe endpointy:

Logowanie/Rejestracja: Zamień wywołania `/api/auth/register` i `/api/auth/login` na rzeczywiste endpointy NestJS (np. `POST /auth/register` itp., lub rozważ użycie NextAuth jeśli planowane). Upewnij się, że po udanej autoryzacji otrzymany token JWT jest przechowywany (np. w `HttpOnly` cookie lub w `localStorage` jeśli używamy go w `fetch`).

Pobieranie profilu twórcy: Funkcja `fetchCreatorData` powinna zostać zastąpiona zapytaniem do API (`GET /creators/{username}`), a mechanizm Next.js (SSG/SSR) może wykorzystać `generateStaticParams` (jeśli lista twórców jest znana) lub `fallback`.

Wysłanie napiwku: Endpoint `/api/tips` wywoływany w `TipModal` powinien zostać obsłużony przez backend (realizujący logikę opisaną w module `Payments`). Upewnij się, że przekazujesz niezbędne dane (ID twórcy, kwota, ewentualnie id metody) i że użytkownik jest uwierzytelniony (JWT).

Wypłaty: Podstrona Wypłaty (`withdrawals`) powinna wywoływać np. `GET /wallet/balance` (dla salda) oraz `POST /wallet/withdraw` przy zleceniu wypłaty. Te endpointy mapują się na integrację z API Circle w backendzie.

Statystyki: W sekcji statystyk obecnie dane są symulowane. Podłącz odpowiednie endpointy (np. `GET /tips/stats`) które zwracają sumy, listy itd. Wykres można zasilić zwracającym dane agregowane (np. sumy dzienne za ostatnie 30 dni).

6. Tailwind i styl: Projekt korzysta z Tailwind, więc aby dostosować branding, edytuj `tailwind.config.js` (kolory, fonty). Pamiętaj o uruchomieniu procesu budowania (np. `npm run dev`) aby wygenerować style. W razie potrzeby dostosuj również komponenty shadcn (pliki w `components/ui/*` jeśli są generowane).

7. Deploy: Aplikację można zbudować komendą `npm run build`. Wynikowa aplikacja (Next.js) może zostać wdrożona na Vercel lub innym hostingu obsługującym Node.js. Upewnij się, że zmienne środowiskowe (np. `API_URL`, klucze OAuth) są ustawione również w środowisku produkcyjnym.

8. Dalsze prace: W trakcie skalowania produktu planowane jest wprowadzenie kolejnych funkcji (np. subskrypcje NFT, AI asystent). Struktura projektu jest przygotowana na łatwe dodawanie nowych podstron i komponentów. Zachowuj konsekwencję w stylowaniu i przestrzegaj zasad design systemu przedstawionych powyżej – dzięki temu TipJar+ utrzyma spójny i profesjonalny wizerunek nawet przy rozbudowie o nowe moduły.

Na koniec, upewnij się, że cały interfejs jest zgodny z założeniami dostępności i wydajności: testuj kontrasty, nawigację klawiaturą, oraz monitoruj obciążenie (np. użycie React.Profiler lub narzędzi Lighthouse dla performance). Dzięki powyższym wytycznym i przykładowemu kodowi, TipJar+ ma solidne fundamenty front-endowe do dalszego rozwoju, łącząc nowoczesne technologie Web3 z przyjaznym UX/UI. Powodzenia z wdrożeniem!